Network Time Protocol (NTP)

จัดทำโดย

นาย ณฐนรร ยังพะกูล

รหัสนักศึกษา 6405500288

เสนอ

อาจารย์นิพิฐ สง่ามั่งคั่ง

รายงานนี้เป็นส่วนหนึ่งของการเรียนวิชา

Computer Network (COS3106)

มหาวิทยาลัยรามคำแหง

คำนำ

ในโลกยุคสมัยนี้เต็มไปด้วยการติดต่อสื่อสารกันอย่างรวดเร็ว โดยเทคโนโลยีต่างๆ หนึ่งในนั้นคือ อินเตอร์เน็ต (The Internet) ที่ไม่ว่าใคร อยู่ที่ไหน ถ้ามีคอมพิวเตอร์ หรือ อุปกรณ์อิเล็กทรอนิกส์ที่เชื่อมต่ออินเตอร์เน็ต ก็สามารถใช้เพื่อติดต่อกันได้ แต่อุปกรณ์เหล่านี้ อาจไม่สามารถทำงานร่วมกันได้อย่างถูกต้อง ถ้าพวกมันเข้าใจเวลาที่ไม่ตรงกัน เพราะฉะนั้น Network Time Protocol (NTP) จึงเข้ามาช่วยประสานเวลาระหว่างอุปกรณ์ให้ตรงกัน

ในรายงานเล่นนี้จะมาอธิบายการทำงานของ Protocol นี้ และเป็นส่วนหนึ่งของวิชา Computer Network (COS3104) หากมีข้อแนะนำ หรือ ข้อผิดพลาดประการใด ผู้จัดทำขอน้อมรับไว้และขออภัยมา ณ ที่นี้ด้วย

ณฐนรร ยังพะกูล (6405500288)

สารบัญ

คำนำ 2

สารบัญ 3

หลักการและทฤษฎีของ Network Time Protocol 4

- อะไรคือ NTP? 4
- NTP กับ Strata 4
- ข้อมูลเวลา 5
- อัลกอริทึมที่ใช้ประสานเวลา 5

ผลการวิเคราะห์การทำงาน เทคนิควิธีการ และมาตราฐานที่เกี่ยวข้อง 7

- รูปแบบข้อความ (Message Format) 7
- วิธีการรับส่งข้อมูลเวลาของ NTP 8
- ตัวอย่างการทำงานของ NTP 9
- รายละเอียดและวิธีการเขียนชุดคำสั่งด้วยภาษาโปรแกรม Python 10

ข้อกังวลเรื่องความปลอดภัยของ NTP 16

แนวทางในการประยุกต์ใช้ และการพัฒนาต่อ 17

แหล่งที่มาและแหล่งอ้างอิง 18

ภาคผนวก 19

หลักการและทฤษฎีของ Network Time Protocol

อะไรคือ NTP?

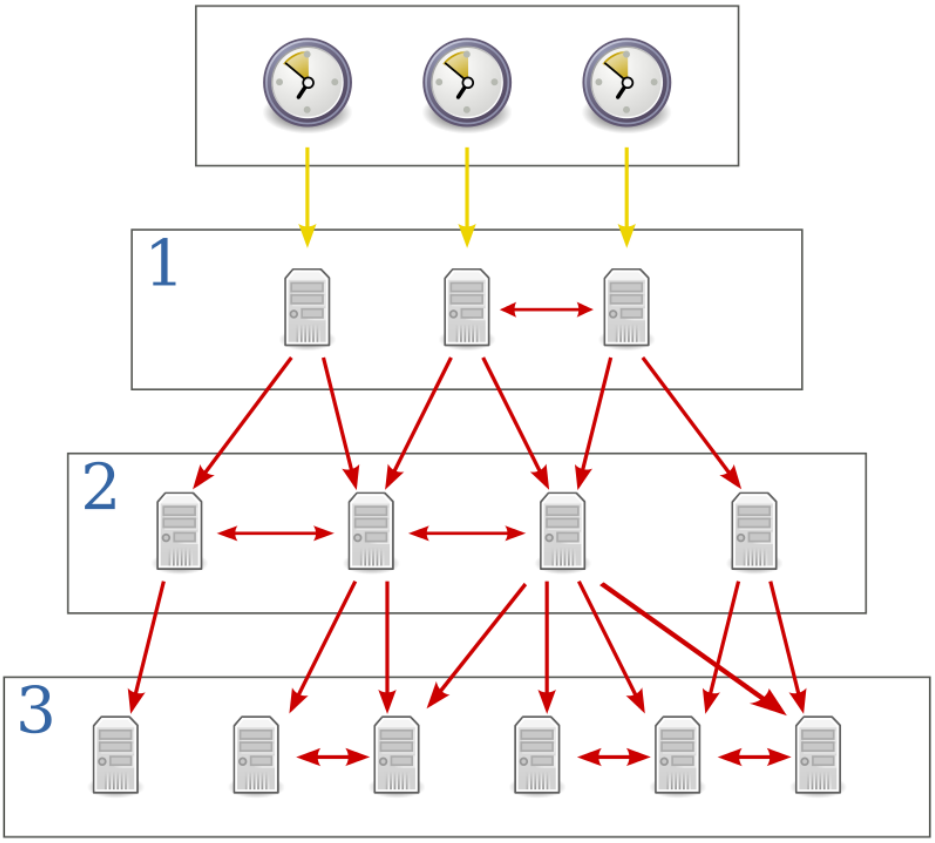
Network Time Protocol หรือ NTP เป็นเครือข่ายโพรโทคอลที่ถูกกำหนดขึ้นมาในปี 1985 เพื่อใช้ประสาน (synchronize) ข้อมูลเวลาระหว่างเครื่องคอมพิวเตอร์ ในระบบเครือข่ายต่างๆ NTP สามารถประสานเวลาได้ในระดับมิลลิวินาที และสามารถกระจายข้อมูลไปตามระบบอินเตอร์เน็ตได้อีกด้วย

โพรโตคอลนี้ใช้การติดต่อสื่อสารระหว่างกันผ่านโมเดลแบบ client-server แต่สามารถปรับให้เป็นแบบ peer-to-peer ได้ด้วยเช่นกัน การทำแบบนี้สามารถทำให้ หน่วยคอมพิวเตอร์ทั้งสองสามารถแชร์ข้อมูลเวลาระหว่างกันได้

NTP ใช้การรับส่งข้อมูลเวลาผ่าน User Datagra Protocol (UDP) ด้วยหมายเลขพอร์ท 123 ปัจจุบันเราใช้เวอร์ชั่น 4 ของโพรโทคอลนี้อยู่

NTP กับ Strata

NTP ใช้โครงสร้างลำดับชั้นที่เรียกว่า Stratum (รูปพหูพจน์คือ Strata) เพื่อใช้เป็นแหล่งของข้อมูลเวลา โดยในแต่ละลำดับจะมีการใช้งานที่แตกต่างกันออกไป



- Stratum 0 เป็นเครื่องจับเวลาที่มีความละเอียดสูง อย่างเช่น นาฬิกาอะตอม หรือระบบดาวเทียม (เช่น GPS) เป็นต้น stratum นี้เป็นต้นกำเนิดของเวลาใน stratum ลำดับอื่นๆ จึงได้ถูกเรียกว่า นาฬิกาอ้างอิง (reference clocks)
- Stratum 1 เป็นเครื่องคอมพิวเตอร์ที่ประสานเวลาของมันกับ stratum 0 โดยตรง ซึ่งเวลาในการประสานจะอยู่ในความละเอียดที่ระดับไมโครวินาที (microseconds) เครื่อง stratum ละดับนี้จึงถูกเรียกว่าเครื่องเซิร์ฟเวอร์หลัก(primary time servers)
- Stratum 2 ถึง 15 เป็นเครื่องคอมพิวเตอร์ที่ประสานกับ Stratum ในลำดับก่อนหน้า เช่น Stratum 2 ประสานกับ Stratum 1 และ Stratum 3 ประสานกับ Stratum 2 ต่อไปเรื่องๆจนถึง Stratum 15 ประสานกับ Stratum 14 เป็นต้น

stratum 16 และ strata ลำดับถัดๆจะถือว่า strata เหล่านั้นไม่มีการประสานเวลาที่ถูกต้อง (unsynchronized)

ข้อมูลเวลา

ข้อมูลเวลาจะถูกรับส่งในรูปแบบ Binary จำนวน 64-bit โดยที่ 32 บิทแรก จะเป็นข้อมูลวินาที และ 32 บิทหลังจะเป็นข้อมูลเศษส่วนของวินาที นั้นหมายถึงว่า ระบบนี้สามารถเก็บข้อมูลวินาทีได้สูงสุด $2^{32} - 1$ วินาที (~136 ปี) และมีความละเอียดสูงสุดในระดับ 2^{-32} วินาที (~233 พิโควินาที)

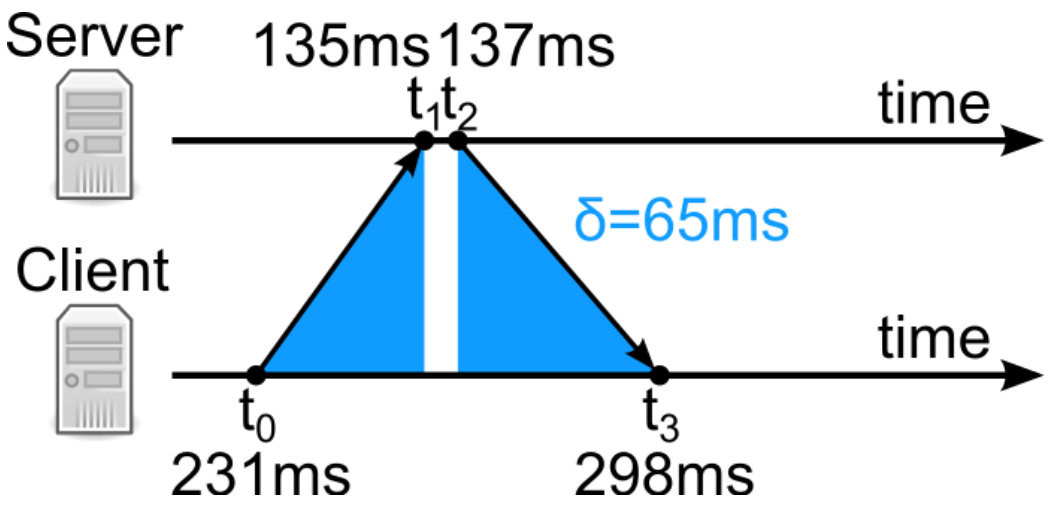
NTP กำหนดจุดเริ่มต้นของเวลา (วินาทีที่ 0) เป็นวันที่ 1 มกราคม ค.ศ. 1900 ดังนั้นวันที่มันสามารถเก็บได้สูงสุดก่อนที่จะวนกลับคือ 7 กุมภาพันธ์ ค.ศ. 2036

อัลกอริทึมที่ใช้ประสานเวลา (Clock synchronization algorithm)

เนื่องจากว่าการรับส่งข้อมูลเวลาเพื่อการประสานระหว่างคอมพิวเตอร์นั้น ต้องทำผ่านระบบเครือข่าย ซึ่งทำให้เกิดความล่าช้าขึ้นได้ ฉะนั้นเพื่อจัดการกับปัญหานี้ NTP ได้กำหนดวิธีการคำนวนเพื่อหาระยะเวลาที่ใช้รับส่งได้ดังนี้

$$\delta = (t_3 - t_0) - (t_2 - t_1) \qquad \text{โดยที่}$$

- t_0 หมายถึงเวลาที่ client ร้องขอไปยัง server
- t_1 หมายถึงเวลาที่ server รับการขอจาก client
- t_2 หมายถึงเวลาที่ server ตอบกลับไปหา client
- t_3 หมายถึงเวลาที่ client ได้รับการตอบกลับของ server



นอกจากนี้ NTP ยังกำหนดวิธีการคำนวนหาจำนวนชดเชยเวลา (Time offset) ได้อีกด้วย ซึ่งถูกกำหนดได้ตามนี้

$$\theta = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}$$

จำนวนชดเชยเวลา (θ) คือจำนวนของเวลาบน cilent ที่แตกต่างไปจากเวลาบน server ถ้า θ เป็นบวก นั้นหมายถึงเวลาบน client เดินล่าช้ากว่าเวลาบน server หรือถ้า θ เป็นลบ นั้นหมายถึงเวลาบน client เดินเร็วกว่าเวลาบน server เป็นต้น

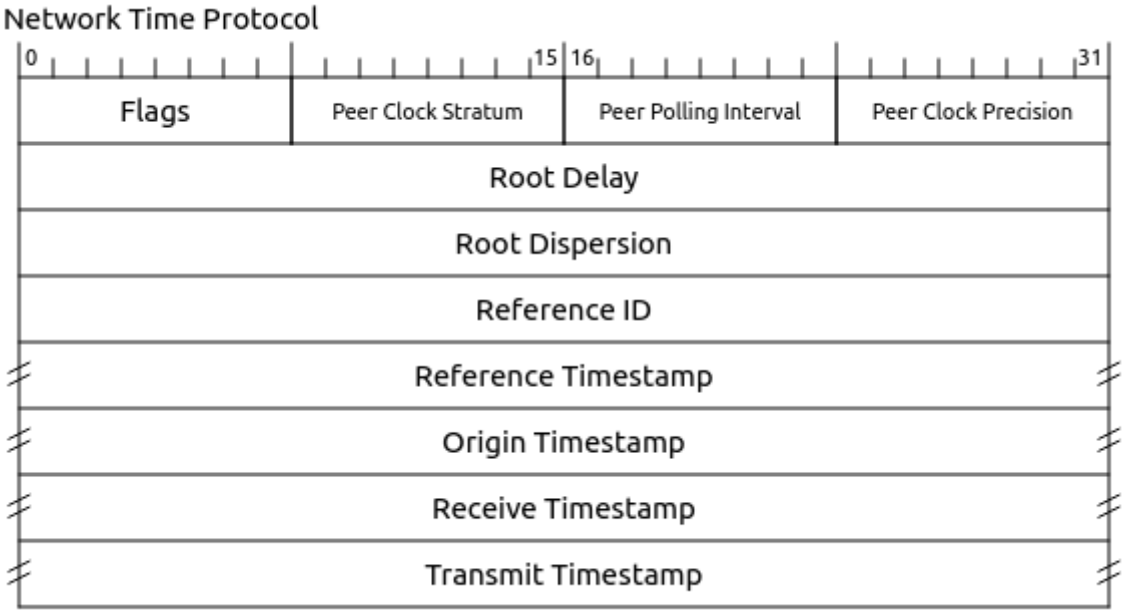
ผลการวิเคราะห์การทำงาน เทคนิควิธีการ และมาตราฐานที่เกี่ยวข้อง

เนื่องจากว่า NTP เป็นโพรโทคอลที่มีความซับซ้อนและเต็มไปด้วยรายละเอียดต่างๆ ที่ผู้ใช้ส่วนใหญ่ไม่จำเป็นต้องใช้ เพราะฉะนั้น โพรโทคอลที่เรียกว่า Simple Network Time Protocol (SNTP) จึงได้ถูกกำหนดขึ้น เพื่อลดความซับซ่อนที่เกิดขึ้นของโพรโทคอลก่อนหน้านี้ และเนื่องจากว่า SNTP ไม่ได้ถือว่าเป็นโพรโทคอลใหม่ และมันยังเป็นสับเซต (subset) ของ NTP มันจึงสามารถใช้งานร่วมกับ NTP ได้

เพราะว่า SNTP มีการใช้งานที่ง่าย ทำให้มันถูกนำเอาไปใช้งานอย่างแพร่หลายอีกด้วย ตัวอย่างที่จะนำมาแสดงให้ดูนั้น จะใช้ SNTP เป็นหลัก

รูปแบบข้อความ (Message Format)

ทั้ง NTP และ SNTP ใช้ User Datagram Protocol (UDP) ในการรับส่งข้อมูลเวลา และหมายเลขพาร์ท (port) ที่ใช้คือ 123



ภาพที่แสดงให้เห็นด้านบนนี้จะเป็นรูปแบบข้อความของ NTP และ SNTP ซึ่งจะถูกส่งต่อจากส่วนหัวของข้อความใน UDP โดยแต่ละฟิลด์ (field) มีความหมายโดยสังเขป ดังนี้ (หมายเลขในลงเล็บเป็นจำนวนบิทที่ใช้)

- Flags (8): แบ่งออกได้เป็นสามฟิลด์ย่อย ดังนี้
    - Leap Indicator (2): ใช้เพื่อบอกว่านาทีสุดท้ายของวันเป็นวินาที leap หรือไม่
        - 0 = ไม่มี
        - 1 = leap +1 วินาที
        - 2 = leap -1 วินาที
        - 3 = นาฬิกาไม่มีการประสาน
    - Version Number (3): เวอร์ชั่นของโพโทคอล ปัจจุบันคือ เวอร์ชั่น 4
    - Mode (3): โดยทั่วไปจะใช้อยู่แค่ 2 โหมดคือ
        - 3 = เพื่อบอกว่าข้อความส่งมาจาก client
        - 4 = เพื่อบอกว่าข้อความส่งมาจาก server
- Peer Clock Stratum (8): บอกถึงชั้นลำดับของ Stratum
- Peer Polling Interval (8): บอกถึงว่า client ไม่ควรจะร้องขอข้อมูลเวลาบ่อยแค่ไหน
- Peer Clock Precision (8): บอกถึงความละเอียดของข้อมูลเวลาที่ส่งไปหา client
- Root Delay (32): บอกถึงจำนวนเวลาที่ล่าช้าไปจาก server นี้กับ server หลัก
- Root Dispersion (32): บอกถึงจำนวนเวลาที่อาจผิดพลาดไปของเวลาบน server
- Reference ID (32): บอกถึงแหล่งข้อมูลของเวลาว่ามาจากแหล่งข้อมูลอะไร หรือประเภทใด
- Reference Timestamp (64):บอกถึงเวลาล่าสุดที่เวลาบน server ได้ทำการอัพเดต
- Origin Timestamp (64): บอกถึงเวลาตอนที่ client ได้ส่งข้อความไปหา server
- Receive Timestamp (64): บอกถึงเวลาตอนที่ client หรือ server ได้รับข้อความจากอีกฝ่าย
- Transmit Timestamp (64): บอกถึงเวลาตอนที่ client หรือ server ได้รับส่งข้อความไปหาอีกฝ่าย

วิธีการรับส่งข้อมูลเวลาของ NTP

เพราะว่าโดยปกติ NTP ใช้โมเดลการสื่อสารแบบ client-server ฉะนั้น การร้องขอข้อมูลเวลาจะต้องเริ่มจากคอมพิวเตอร์ client โดยที่ client จะห่อรูปแบบข้อความที่อธิบายในหัวข้อก่อนหน้า ลงในโพรโทคอล UPD แล้วส่งไปให้ server บน port 123

ถ้าใช้ SNTP ในการส่ง client ไม่ต้องทำอะไรมาก เพียงแต่กำหนดฟิลด์ Version Number เป็น 4, Mode เป็น 3 (client) และ Transmit Timestamp เป็นเวลาปัจจุบัน แล้วในฟิลด์อื่นๆ client สามารถกำหนดค่าที่เหลือเป็น 0 ได้

ในส่วนการส่งของ server ควรมีรูปแบบข้อความที่เป็นไปตามคำอธิบายในหัวข้อก่อนหน้า เช่น Mode ควรเป็น 4 (server), Peer Clock Stratum ควรมีค่าเป็น 1 ถึง 15 (ขึ้นอยู่กับแต่ละ server) เป็นต้น

ตัวอย่างการทำงานของ NTP

ในทุกระบบปฏิบัติการ (Operating System - OS) สมัยนี้ได้นำเอา NTP ไปใช้เพื่ออัพเดตหรือประสานข้อมูลเวลาของเครื่องคอมพิวเตอร์อยู่เสมอ ทำให้คอมพิวเตอร์สื่อสารระหว่างกันได้อย่างไม่มีปัญหา

ตัวอย่างที่จะแสดงให้เห็นต่อไปนี้เป็น รูปแบบข้อความที่ NTP ได้ทำการรับส่งระหว่างเครื่องคอมพิวเตอร์ของผู้ใช้ (client) และ โดเมน ntp.ubuntu.com (server) ซึ่งถูกดักจับด้วยโปรแกรม Wireshark ในขณะที่ OS ทำการอัพเดตเวลาของเครื่อง

<u>Client to Server Data Frame</u>

```
23 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 e8 83 78 ab 3b 88 06 d9
```

<u>Server to Client Data Frame</u>

```
24 02 00 f7 00 00 00 4c 00 00 00 1b c9 44 58 6a
e8 83 77 cc b2 e7 f7 93 e8 83 78 ab 3b 88 06 d9
e8 83 78 ac 23 7c 89 f6 e8 83 78 ac 23 7d a3 a3
```

ข้อมูลถูกแสดงในรูปแบบ Hex จำนวน bytes ในการรับส่งแต่ละครั้งอยู่ที่ 48 bytes โดยจะสังเกตเห็นได้ว่าข้อมูลจาก client ไปยัง server โดยส่วนใหญ่จะมีค่าเป็น 00 ยกเว้นแต่ byte แรก (version number และ mode) และ 8 bytes สุดท้าย (transmit timestamp) ซึ่งตรงกับที่ได้อธิบายไว้ในหัวข้อก่อนหน้านี้

ในส่วนของ server ไปยัง client เราสามารถ decode ข้อมูลออกมาได้ดังนี้

- 0x24 = 0b00100100: 2 บิทแรกบอกว่าไม่มี leap seconds 3 บิทต่อมาบอกว่า NTP นี้เป็นเวอร์ชั่น 4 และ 3 บิทสุดท้ายบอกว่า ข้อมูลถูกส่งมาจาก server
- 0x02 = 2: บอกถึง server เป็น stratum ลำดับชั้นที่ 2 (secondary time server)

- 0x00: บอกว่าไม่มีเวลาที่ต้องรอในการร้องขอข้อมูลซ้ำ หรือไม่ได้ระบุ
- 0xf7 = -9: มีความละเอียดเป็น $2^{-9} = 0.001953125$ วินาที
- 0x0000 = 0, 0x004c = 76: server มีความล่าช้าเป็น $0 + \frac{76}{2^{16}} \approx 0.00116$ วินาที
- 0x0000=0,0x001b=27:มีโอกาศผิดพลาดมากที่สุดที่ $0 + \frac{27}{2^{16}} \approx 0.000412$ วินาที
- 0xc9, 0x44, 0x58, 0x6a: แหล่งข้อมูลเวลามาจาก IPv4: 201.68.88.106
- 0xe88377cc = 3900929996, 0xb2e7f793 = 3001546643:
    - $3900929996 + \frac{3001546643}{2^{32}} \approx 3900929996.698852036$ วินาที
    - Reference Timestamp: Aug 13, 2023 15:39:56.698852036 UTC
- 0xe88378ab = 3900930219, 0x3b8806d9 = 998770393:
    - $3900930219 + \frac{998770393}{2^{32}} \approx 3900930219.232544353$ วินาที
    - Origin Timestamp: Aug 13, 2023 15:43:39.232544353 UTC
- 0xe88378ac = 3900930220, 0x237c89f6 = 41212642195:
    - $3900930220 + \frac{41212642195}{2^{32}} \approx 3900930220.138619062$ วินาที
    - Receive Timestamp: Aug 13, 2023 15:43:40.138619062 UTC
- 0xe88378ac = 3900930220, 0x237da3a3 = 595436451:
    - $3900930220 + \frac{595436451}{2^{32}} \approx 3900930220.138635852$ วินาที
    - Transmit Timestamp: Aug 13, 2023 15:43:40.138635852 UTC

เราสามารถใช้สมการต่อไปนี้เพื่อหาความล่าช้าในการรับส่งได้ดังนี้

$$\delta = (t_3 - t_0) - (t_2 - t_1) \qquad \text{โดยที่}$$

- t_0 = Origin Timestamp = 3900930219.232544353
- t_1 = Receive Timestamp = 3900930220.138619062
- t_2 = Transmit Timestamp = 3900930220.138635852
- t_3 = เป็นเวลาที่ client ได้รับการตอบกลับจาก server สมมุติว่า = 3900930220.953471568

ดังนั้นเวลารวมที่ใช้ในการรับส่งข้อมูล δ จะมีค่าเป็น 1.720910425 วินาที

รายละเอียดและวิธีการเขียนชุดคำสั่งด้วยภาษาโปรแกรม Python

จากหัวข้อก่อนหน้า เราเห็นตัวอย่างการทำงานที่ OS ใช้ NTP แล้ว ในหัวข้อนี้เราจะนำไปประยุกต์ใช้ในโปรแกรมภาษา Python โดยเราจะเขียนการส่ง และรับข้อมูลในระดับของโพรโทคอล

เริ่มแรก เราควร `import` โมดุลที่เราจะต้องใช้ก่อน

```
import socket, struct, secrets, datetime, time
```

จากนั้นให้สร้างชุดข้อมูลว่างจำนวน 48 bytes เพื่อใช้ในการส่ง

```
query = bytearray(48)
```

กำหนดค่าลงในข้อมูลที่เราสร้างขึ้น ตามที่ระบุไว้ในหัวข้อก่อนหน้านี้

```
query[0] = 0x23  # version=4, mode=3 (client)
query[40:48] = secrets.token_bytes(8)
```

สังเกตว่า 8 bytes สุดท้ายเราใช้ข้อมูลสุ่ม ในที่นี้ เนื่องจากว่า server ไม่ได้ทำอะไรกับข้อมูลนี้ เพียงแค่คัดลอกและส่งข้อมูลเดิมกลับ เราสามารถใช้ข้อมูลนี้ในการยืนยันความถูกต้องได้

ต่อมาเราจะสร้าง `socket` เพื่อใช้ในการส่งข้อมูล

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

ในส่วนของ `socket.AF_INET` และ `socket.SOCK_DGRAM` หมายถึง เราจะใช้ UDP บน IPv4

ต่อมาเราจะหาว่า NTP server โดเมน pool.ntp.org นี้มี IP อะไร

```
addr = socket.getaddrinfo(
    'pool.ntp.org', 'ntp', socket.AF_INET, socket.SOCK_DGRAM)[0][-1]
```

ค่า `addr` ตอนนี้จะมี IP และ พอร์ทของ NTP (123) เก็บอยู่

ตอนนี้เราพร้อมที่จะส่งข้อมูลแล้ว

```
try:
    s.settimeout(1)
    local_t1 = time.time()
    assert s.sendto(query, addr) == 48
    data = s.recv(48)
    local_t2 = time.time()
finally:
    s.close()
```

ตัวแปล `local_t1` และ `local_t2` เก็บค่าข้อมูลเวลาส่งออกและรับเข้าข้อมูลตามลำดับ

ตอนนี้ตัวแปร `data` มีค่าข้อมูลเวลาที่ server ได้ส่งกลับมาให้เรา ต่อไปเรานำค่านี้มาแตกออกเพื่อจะได้จัดการต่อไปได้ง่าย

```
x = struct.unpack('!BBBbiIIQQQQ', data)
leap_ind = x[0] >> 6  # Leap Indicator: 0=no warning, 1=high, 2=low,
3=alarm
version = (x[0] >> 3) & 0x7  # Version Number: current=4
mode = x[0] & 0x7            # Mode: 3=client, 4=server
strat = x[1]     # Stratum
poll = x[2]      # Poll Interval
prec = x[3]      # Precision
r_delay = x[4]   # Root  Delay
r_disp = x[5]    # Root  Dispersion
ref_id = x[6]    # Reference Identifier
ref_t = x[7]     # Reference Timestamp
orig_t = x[8]    # Originate Timestamp
recv_t = x[9]    # Receive Timestamp
tran_t = x[10]   # Transmit Timestamp
```

เพื่อความถูกต้อง เราควรตรวจสอบข้อมูลที่เราได้รับมา

```
assert leap_ind != 3, 'alarm condition (clock not synchronized)'
assert version == 4, 'version number does not match the request\'s'
assert mode == 4, 'expected mode: server'
assert 0 <= strat < 15, 'invalid stratum'
assert orig_t.to_bytes(8, 'big') == query[40:48]
```

ถ้าไม่มี errors เกิดขึ้น แสดงว่าทุกอย่างถูกต้อง

ตอนนี้เราพร้อมที่จะแสดงข้อมูลที่เราได้รับมาแล้ว

```
if leap_ind == 1:
   print('Leap Indicator: last minute has 61 seconds')
elif leap_ind == 2:
   print('Leap Indicator: last minute has 59 seconds')
```

ถ้า server ส่งข้อมูล leap วินาทีมา มันจะ print ค่าออกมาให้เห็น

แสดงเวอร์ชั่นของ NTP ได้ตามนี้

```
print(f'Version Number: {version}')
```

แสดงหมายเลข stratum

```
print(f'Stratum: {strat} ', end='')
match strat:
   case 0:
       print('(kiss-o\'-death)')
   case 1:
       print('(primary reference)')
   case _:
       print('(secondary reference)')
```

ถ้าเป็น 0 หมายถึงเราเรียกร้องขอข้อมูลมากเกินไป 1 หมายถึง server นี้เป็น server หลัก และอื่นๆคือ server รอง

```
print(f'Poll Interval: {2**poll} s')
print(f'Precision: {2**prec * 1e6:.6f} us')
print(f'Root Delay: {r_delay/2**16:.6f} s')
print(f'Root Dispersion: {r_disp/2**16 * 1e3:.6f} ms')
```

แสดงระยะเวลาที่ server ต้องการให้ร้องขอเป็นอย่างน้อย ความละเอียดของเวลา เวลาที่ล่าช้า และเวลาที่อาจผิดพลาดตามลำดับ

แสดงตัวระบุอ้างอิง หรือแหล่งที่มาของข้อมูล

```
print(f'Reference Identifier: ', end='')
rid = ref_id.to_bytes(4, 'big')
if strat < 2:
   rid = rid.replace(b'\x00', b'').decode()
   print(f'{rid} ({RID_CODES.get(rid)})')
else:
   rid = '.'.join([str(rid[i]) for i in range(3, -1, -1)])
   print(rid)
```

สำหรับ stratum 1 ค่านี้จะเป็นตัวย่อ 3 ถึง 4 ตัวอักษร (`RID_CODES` ระบุไว้ที่ภาคผนวก) และอื่นๆจะเป็นเลข IPv4 Address

ค่าข้อมูลเวลาแสดงดังนี้

```
print(f'Reference Timestamp: {ref_t/2**32}')
print(f'Receive Timestamp: {recv_t/2**32}')
print(f'Transmit Timestamp: {tran_t/2**32}')
```

บอกถึงเวลาที่ server อัพเดตข้อมูลล่าสุด เวลาที่ server รับการร้องขอ และ เวลาที่ server ส่งข้อมูลไปยัง client ตามลำดับ

ตัวอย่าง output ของคำสั่ง print อาจเป็นดังนี้

```
Version Number: 4
Stratum: 2 (secondary reference)
Poll Interval: 16 s
Precision: 0.029802 us
Root Delay: 0.005585 s
Root Dispersion: 0.671387 ms
Reference Identifier: 17.59.147.203
Reference Timestamp: 3900976538.558348
Receive Timestamp: 3900977063.0048466
Transmit Timestamp: 3900977063.0048866
```

ก่อนที่เราจะคำนวนหาค่าความล่าช้าของการรับส่งข้อมูล เราจำเป็นต้องปรับค่าข้อมูลเวลาที่ server ส่งมาให้เราให้ถูกต้องกับระบบของ Python ก่อน

```
JAN_1900 = datetime.datetime(1900, 1, 1)
recv_dt = JAN_1900 + datetime.timedelta(seconds=recv_t/2**32)
tran_dt = JAN_1900 + datetime.timedelta(seconds=tran_t/2**32)
```

เนื่องจากว่า วินาทีที่ 0 ของ NTP คือปี 1900 แต่ Python ใช้ปี 1970 เราจึงต้องบังคับให้ Python มันใช้ปี 1900 ตามที่แสดงอยู่นี้

ตอนนี้เราพร้อมแล้วที่จะแสดงข้อมูล

```
print(f"UTC: {tran_dt.strftime('%c')}")
```

แสดงวันเดือนปี และเวลาตอนที่ server ส่งข้อมูลมาให้เราในระบบ UTC

```
print(f'Server latency: {(tran_t - recv_t)/2**32 * 1e3:.6f} ms')
```

แสดงระยะเวลาที่ server ประมวลผลข้อมูลเวลาก่อนที่มันจะส่ง

```
rt_delay = (local_t2 - local_t1) - (tran_t - recv_t)/2**32
print(f'Roundtrip delay: {rt_delay * 1e3:.3f} ms')
```

แสดงจำนวนเวลาล่าช้าของการรับส่งข้อมูล

```
print(f'Last update: {(tran_t - ref_t)/2**32:.3f} seconds ago')
```

แสดงเวลาว่านานแค่ไหนแล้วที่ server นี้ได้ทำการอัพเดตข้อมูลเวลา

```
offset = ((recv_dt.timestamp() + 25200 - local_t1) +
(tran_dt.timestamp() + 25200 - local_t2)) / 2
print(f'System clock offset (7hr. adjusted): {offset * 1e3:.3f} ms')
```

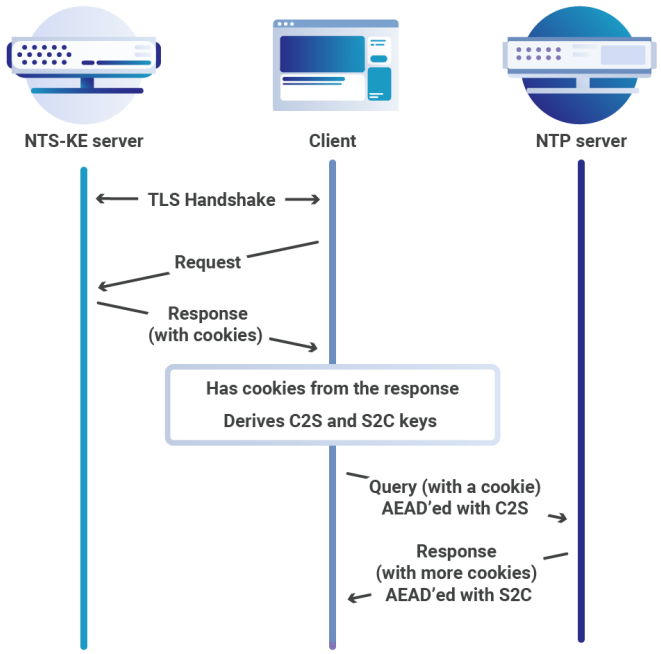
แสดงจำนวนเวลาที่ client เกิดการคลาดเคลื่อนไปจากเวลาของ server เนื่องจากว่า `local_t1` และ `local_t2` มี time zone offset เป็น 25200 วินาที เราจำเป็นต้องปรับเพิ่มค่านี้ให้กับ `recv_dt` และ `tran_dt` ด้วย

ตัวอย่าง output ของคำสั่ง print อาจเป็นดังนี้

```
UTC: Mon Aug 14 04:44:23 2023
Server latency: 0.040401 ms
Roundtrip delay: 9.184 ms
Last update: 524.447 seconds ago
System clock offset (7hr. adjusted): 67.809 ms
```

ข้อกังวลเรื่องความปลอดภัยของ NTP

Network Time Protocol (NTP) เป็นโพรโทคอลที่รับส่งข้อมูลผ่าน User Datagram Protocol (UDP) อีกทีหนึ่ง โดยการรับส่งนี้เป็นการกระทำแบบ ไม่มีความน่าเชื่อถือ (authentication) เพราะฉะนั้นแล้ว Server ที่ใช้ NTP อาจเสี่ยงต่อการโจมตีคนตรงกลาง (Man-In-The-Middle - MITM) นั้นหมายถึงว่า ระบบของผู้โจมตีจะสามารถดักจับและเปลี่ยนข้อมูลเวลาของ Servers ใดๆที่ส่งไปยัง Clients ได้อย่างง่ายดาย โดยผลกระทบที่ตามมามีหลายรูปแบบ เช่น บังคับให้กุญแจเข้ารหัส (cryptographic key) หมดอายุ เป็นต้น เพื่อเป็นการป้องกันการโจมตีชนิดนี้ Network Time Security (NTS) จึงถูกการกำหนดขึ้น ซึ่งมาตรฐานนี้จะใช้ Transport Layer Security (TLS) ในการเข้ารหัส โดยมีวิธีการคร่าวๆ ดังนี้



การร้องของจะแบ่งออกเป็นสองช่วง ร้องขอกุญแจเข้ารหัสกับ NTS-KE server และใช้กุญแจ โดยการใช้กุญแจนั้น client จะต่อท้ายข้อมูลเวลาด้วยข้อมูลของ
กุญแจ และส่งข้อมูลนั้นไปยัง NTP server ที่รองรับการเข้ารหัส เมื่อ NTP server ตอบกลับ จะมีข้อมูลพิเศษเพิ่มเข้ามา ทำให้เรารู้ได้ว่าข้อมูลนั้นมีการเปลี่ยนแปลงหรือไม่

นอกจากการโจมตีแบบคนตรงกลางแล้ว NTP Server ยังสามารถใช้เป็นอาวุธเพื่อโจมตีคอมพิวเตอร์อื่นๆได้ด้วย การโจมตีนี้เรียกว่า การโจมตีโดยปฏิเสธการให้บริการ (DDoS) โดยวิธีการนี้ ผู้โจมตีจะส่งข้อมูลจำนวนเล็กๆไปให้กับ Server ที่ให้บริการ NTP โดยที่หมายเลข IP ที่ส่งกลับจะถูกแก้ไขเป็นหมายเลขของเป้าหมาย (ที่ต้องการโจมตี) ข้อมูลที่ NTP Server ส่งไปหาเป้าหมายมีจำนวนใหญ่กว่ามาก ทำให้ข้อมูลที่ส่งเหมือนกับว่ามันถูกขยายเพิ่มขึ้น จากเครื่องผู้โจมตีไปยังเป้าหมาย วิธีการนี้เรียกว่า การโจมตีแบบขยาย (amplification attack) เพื่อเป็นการป้องกัน เครื่องให้บริการ NTP สามารถกำหนดไม่ให้ มีการเรียกร้องจากภายนอก

แนวทางในการประยุกต์ใช้ และการพัฒนาต่อ

NTP เป็นโพโทคอลที่ใช้กันอย่างแพร่หลาย และมีการใช้งานที่แตกต่างกันออกไป เช่น

- ใช้เพื่อประสานเวลาระหว่างคอมพิวเตอร์บนเครือข่ายเพื่อให้คอมพิวเตอร์เหล่านั้น มีความมั่นคง และเวลาที่ตรงกัน
- ใช้ในเรื่องการเข้ารหัสเพื่อความปลอดภัย เนื่องจากว่าในระบบการเข้ารหัสบางอย่างจำเป็นต้องใช้กุญแจที่มีวันหมดอายุ การที่ระบบต้องรู้เวลาปัจจุบันจึงเป็นเรื่องที่จำเป็น
- การบันทึกข้อมูล (logging) มีความจำเป็นที่จะต้องมีเวลาบันทึกอยู่ด้วย นอกจากจะยังเป็นตัวบ่งบอกถึงช่วงเวลาที่เหตุการณ์ใดๆเกิดขึ้นแล้ว มันยังช่วยให้การตรวจสอบความสัมพันธ์ของเวลาระหว่างระบบ เป็นไปได้อีกด้วย
- ข้อมูลธุรกรรมทางการเงิน จำเป็นมากที่ต้องรู้เวลา เพื่อให้รู้ถึงลำดับของการซื้อขายสิ้นค่าที่เกิดขึ้น
- อุปกรณ์ Microcontroller และ IoT จำเป็นต้องรู้เวลา เพื่อให้การทำงานต่างๆ เช่น เซ็นเซอร์ และระบบการสื่อสาร เป็นไปได้อย่างถูกต้อง

การพัฒนาระบบ NTP เพื่อให้มีความปลอดภัยและประสิทธิภาพที่ดีขึ้นสามารถทำได้ดังนี้

- เลือกแหล่งที่มาของข้อมูลเวลาที่ดี ช่วยลดปัญหาที่อาจเกิดขึ้นในภายหลักได้ เช่น ข้อมูลเวลาที่ไม่ถูกต้อง หรือปัญหาการเชื่อมต่อกับ NTP server นั้นๆ
- เลือกแหล่งของข้อมูลหลายๆแหล่ง ช่วยลดปัญหาขาดแคลนขอข้อมูลได้ หากผู้ให้บริการใดๆไม่สามรถติดต่อได้
- Software ที่ใช้งาน NTP ควรเป็นเวอร์ชั่นล่าสุด การอัพเดตช่วยลดช่องโหว่ที่อาจเกิดขึ้นได้
- เพื่อลดโอกาศการโจมตี Server สามารถเลือกที่จะรับการขอร้องเฉพาะ IP บางส่วนที่เชื่อถือได้ หรือปิดไม่ให้เข้าถึงได้จากภายนอก
- ติดตั้งเครื่องมือที่ติดตามการใช้งานต่างๆของ NTP server เช่น จำนวนการร้องขอข้อมูลในช่วงเวลาๆหนึ่ง เวลาในการประมวลผลข้อมูล เป็นต้น ทั้งนี้เพื่อช่วยให้ผู้ดูแลระบบเข้าใจถึงปัญหาที่อาจตามมาในภายหลักได้

แหล่งที่มาและแหล่งอ้างอิง

RFC4330: https://datatracker.ietf.org/doc/html/rfc4330

NTP Wikipedia Article: https://en.wikipedia.org/wiki/Network_Time_Protocol

Wireshark Network Capture Tool: https://www.wireshark.org

Python Socket Documentation: https://docs.python.org/3/library/socket.html

Network Time Security (NTS): https://blog.cloudflare.com/announcing-cfnts/

ภาคผนวก

```
RID_CODES = {
   'LOCL': 'uncalibrated local clock',
   'CESM': 'calibrated Cesium clock',
   'RBDM': 'calibrated Rubidium clock',
   'PPS': 'calibrated quartz clock or other pulse-per-second source',
   'IRIG': 'Inter-Range Instrumentation Group',
   'ACTS': 'NIST telephone modem service',
   'USNO': 'USNO telephone modem service',
   'PTB': 'PTB (Germany) telephone modem service',
   'TDF': 'Allouis (France) Radio 164 kHz',
   'DCF': 'Mainflingen (Germany) Radio 77.5 kHz',
   'MSF': 'Rugby (UK) Radio 60 kHz',
   'WWV': 'Ft. Collins (US) Radio 2.5, 5, 10, 15, 20 MHz',
   'WWVB': 'Boulder (US) Radio 60 kHz',
   'WWVH': 'Kauai Hawaii (US) Radio 2.5, 5, 10, 15 MHz',
   'CHU': 'Ottawa (Canada) Radio 3330, 7335, 14670 kHz',
   'LORC': 'LORAN-C radionavigation system',
   'OMEG': 'OMEGA radionavigation system',
   'GPS': 'Global Positioning Service'
}
```